GROUP PROJECT
Java +Masters

Task Outline:

You are to make a Java project that utilises REST APIs, and a MySQL database. The requirements are in the next section.

You may choose one of the given topics (at the end of this document), or come up with an idea of your own - the only solid requirements are the code ones in the next section.

Notes:

- You **can combine criteria**
- Note: you **do not have to use the themes**, as long as you meet all of the criteria outlined.
- There is **no front-end** requirement to these.
- The **assignments**, whilst they cannot be used for this project, **strongly reflect elements of the group project**. So, use these as practice for your group project.
- Whilst there are a lot of marks available, these are generous for your implementation of certain technologies. This is not reflective of the size of the project, **quality over quantity**! Try to stick to the minimum requirements to start with, you can tackle stretch goals afterwards if time permits.

Mark Breakdown:

- Documentation (including README) **46 marks**
- Version Control **6 marks**
- Overall application and Java Coding **50 marks**
- REST endpoints **11 marks**
- DB integration **21 marks**
- Testing **23 marks**
- Deployments **19 marks**

Total marks available:

- **176 marks**

Recommended Timeline:

This is a rough timeline, and does not hit all criteria individually - it is only meant as guidance.

Week 1

- Define your project - what is it going to be?
- Make draft flow charts, e.g. using Miro or Figma
- Outline the documentation required
- Create GitHub repositories
- Start designing your class architecture, and potentially some method logic
- Design how you will integrate REST endpoints and a MySQL Database (note, this does not need to be normalised, so make this a loose design until you've covered S8 in week 2)

Week 2

- Copy your code into a Spring Boot project
- Continue working on the logic
- Integrate Lombok to your code
- Create any endpoints
- Design and connect your database

Week 3

- Write unit tests for your classes
- Create a test plan for further testing
- Add a flowchart to your README

Week 4

- Refactoring
- After S14, create a dockerfile and dockercompose.yml for your application
- Finalise documentation

Requirements:

1. Design your application requirements: ***(week 1)***
    a. Within markdown files, in a documentation directory in your project, create documentation that covers:
        i. Your application requirements and a summary of the app itself
        ii. How you will achieve these
        iii. A manual test plan
2. Create a spring boot app: ***(week 2)***
    a. Using the Spring Initializr, or otherwise
3. REST endpoints ***(week 2)***
    a. **Utilise endpoints with appropriate parameters**
        i. Consider response codes and return values
    b. **Create an openAPI spec**, and add the url to this in the README.md
    c. You only get marks awarded for up to two endpoints, you are welcome to add more endpoints in but there is no requirement to.
4. Connecting database ***(week 2)***
    a. **Create a database** in MySQL workbench
        i. Add this code in a .sql file within the project for reference
    b. **Create and initialise your tables** in MySQL workbench
        i. Add this code in a .sql file within the project for reference
    c. Using **JPA**:
        i. Create an **Entity** for your table
        ii. Create a **Repository** for your table
    d. Utilise your repository for database interactions in your program to:
        i. Retrieve data from the table
        ii. Save data to the table

You will not be assessed on your normalisation here, rather focus on making table entries representable as objects - e.g. a 'people' table will have a 'person' entry, which makes sense as an object.

5. Testing ***(week 3)***
    a. Unit test the majority of methods
    b. Mock any external dependencies using Mockito
6. Exceptions ***(week 3)***
    a. Handle exceptions appropriately
7. Docker ***(week 4)***
    a. Create a Dockerfile to create an image of your application
    b. Use your image in a dockercompose.yaml file, alongside one other service
    c. Provide sufficient instructions on how to use these
8. Further Documentation ***(weeks 2 - 4)***
    a. Create a document to propose your deployment plans
        i. This should include a pipeline (can be a diagram, or a a list)
        ii. Detail the stages you will have, including any testing
9. General (Spring) Coding: ***(week 1 onwards)***
    a. Utilise an appropriate class **architecture**
    b. Use constructors appropriately, and cleanly (e.g. using Lombok annotations)
    c. Use logs appropriately, using @SLf4J or otherwise
        i. If you choose to use another method of logging, this will be taken into consideration provided you detail the choice in your documentation
    d. Appropriate use of **Spring Boot configuration**
        i. Store necessary values here, and retrieving appropriately in code
    e. Add to your **README.md** with running instructions
        i. Include information about how to edit configuration as required

Project Ideas:

Transaction Management

- Accepts a transaction via incoming_transaction
    - Has some basic validation (you can make these rules up yourself)
    - Processes these transactions

Bookstore Backend

- Endpoints to provide functionality such as add_book, get_books, make_purchase, refund_purchase, return_book, etc

Name Matching Service

- Have a DB of names
- Expose endpoints to add names, and search for names
    - Have logic to search for potential matches, e.g. partial names, or common shortenings perhaps